

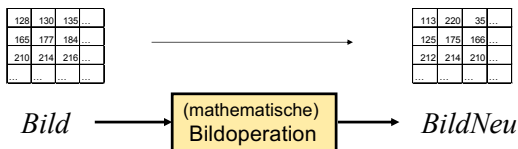
Bildmanipulationen mit Filtern

Prof. Dr. Klaus Jung

Inhalt

- Lineare Filter
 - Formale Eigenschaften
 - Beispiele
- Nichtlineare Filter
 - Beispiele
- Kantendetektion
 - Filter zur Kantendetektion

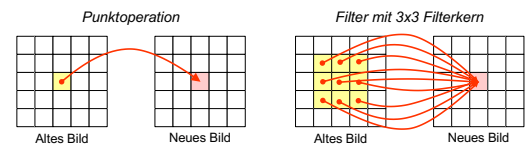
Drei elementare Verfahren der digitalen Bildmanipulation



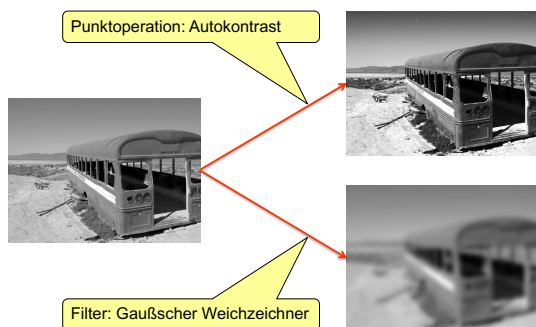
- Bildpunktoperationen
 - angewendet auf ein Bild
 - Überlagerung von zwei oder mehreren Bildern
- Nachbarschaftsoperationen / Filter
- geometrische Operationen

Punktoperationen vs. Filter

- Von *Punktoperationen*
 - Neuer Wert nur von Wert an gleicher Pixel-Position abhängig
 - Kontrast / Helligkeit / Farbe / Gamma
- Zu *Filtern*
 - Neuer Wert abhängig von Werten in Umgebung der Pixel-Position

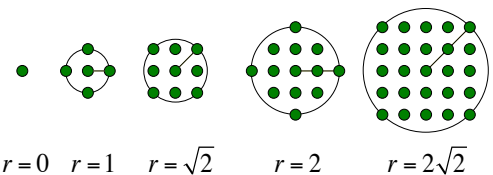


Punktoperationen vs. Filter



Nachbarschaft

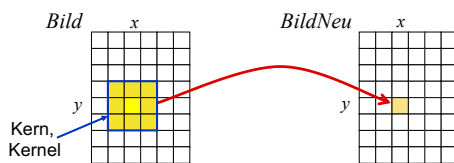
$$N_r(x, y) = \left\{ (i, j) : 0 \leq (i - x)^2 + (j - y)^2 \leq r^2 \right\}$$



$r = 0$ $r = 1$ $r = \sqrt{2}$ $r = 2$ $r = 2\sqrt{2}$

Vierer-
nachbar-
schaft Achter-
nachbar-
schaft

Nachbarschaftsoperatoren = Filter



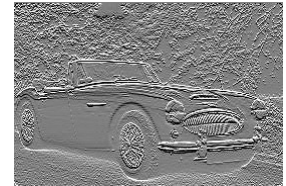
$$PixelNeu(x, y) = fkt(Nachbarschaft\ des\ Pixels(x, y))$$

- Schärfung und Weichzeichnung
- Entstörung
- Kantenbestimmung

Einfaches Beispiel:

Vertikale Reliefbildung

$$PixelNeu(x, y) = 2 \cdot (Pixel(x, y) - Pixel(x, y - 1)) + 128$$



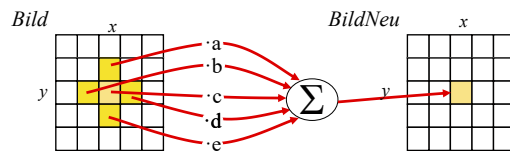
Bild

BildNeu

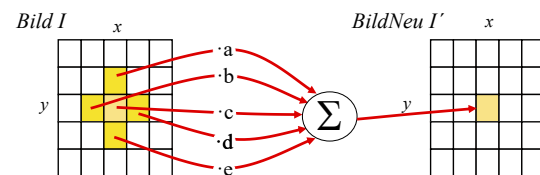
Nachbarschaftsoperatoren / Filter

Lineares Filter:

Jeder (Pixel-)Wert wird durch die linear gewichtete Kombination der (Pixel-)Werte seiner „Nachbarschaft“ (der Kernelregion) ersetzt.



Lineare Filter



Kernel:

$$PixelNeu(x, y) = a \cdot Pixel(x, y - 1) + b \cdot Pixel(x - 1, y) + c \cdot Pixel(x, y) + d \cdot Pixel(x + 1, y) + e \cdot Pixel(x, y + 1)$$

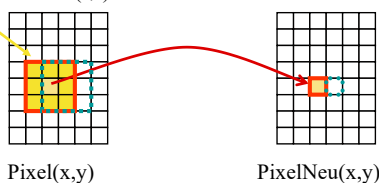


Lineare Filter: Kernel

$$PixelNeu(x, y) = \sum_{\forall k} \sum_{\forall l} H(k, l) \cdot Pixel(x + k, y + l)$$



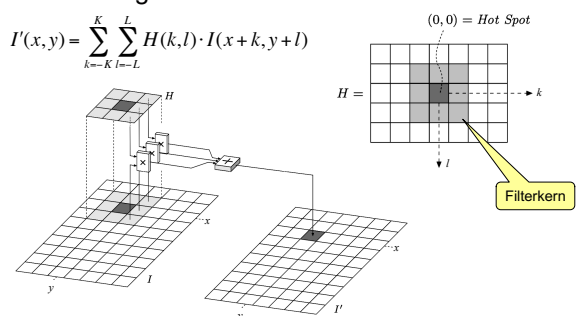
Kern, Kernel $H(k, l)$



Lineare Filter: Kernel

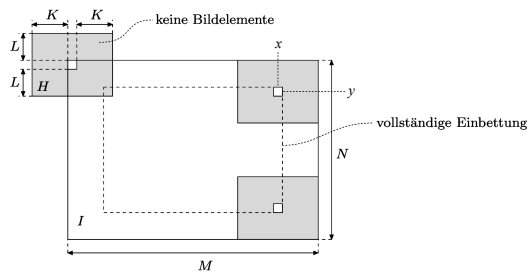
□ Berechnung der neuen Pixel-Werte

$$I'(x, y) = \sum_{k=-K}^K \sum_{l=-L}^L H(k, l) \cdot I(x + k, y + l)$$



Randbehandlung

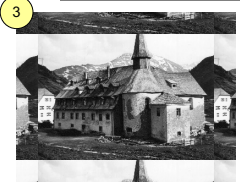
- Was tun, wenn keine Pixel verfügbar?



Randbehandlung

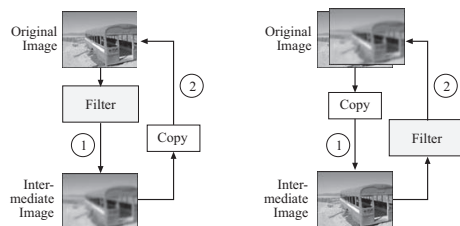
- Pixel außerhalb des Bildes

1. Konstanten Wert setzen
2. Konstant fortsetzen
3. Zyklisch wiederholen
4. Spiegeln wiederholen



Filteroperationen

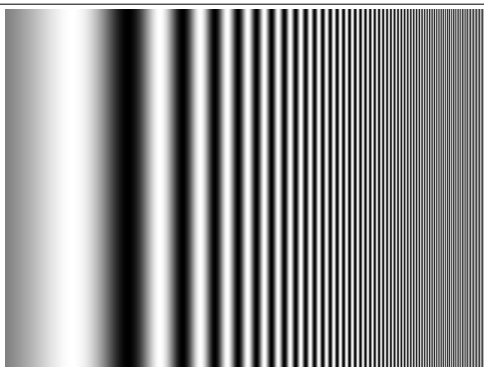
- Wohin mit den berechneten Werten?
 - Punktoperationen: *in place* möglich
 - Filter: Kopie des Bildes notwendig: 2 Varianten:



Wirkungsweise linearer Filter

- Bildpunktoperatoren (Gradationskurven) skalieren Pixel-(werte) d. h. sie verändern Amplitudenverhältnisse
- Lineare Nachbarschaftsoperatoren (Filter) skalieren Spektralbereiche d. h. sie verändern die Frequenzverhältnisse

Bild unterschiedlicher örtlicher Frequenzen



Gefilterte Bilder



ungefiltert



nach Tiefpassfilterung

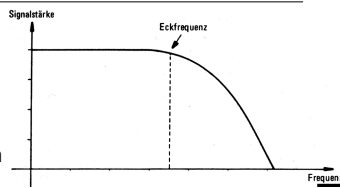


nach Hochpassfilterung

Filter - Charakteristika

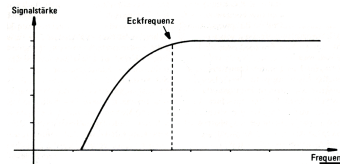
Tiefpass - Filter

Frequenzen oberhalb einer bestimmten Frequenz, der Eckfrequenz bzw. Grenzfrequenz werden abgeschwächt, tiefe Frequenzen können „passieren“



Hochpass-Filter

Frequenzen unterhalb der Eckfrequenz werden abgeschwächt, die hohen können passieren.



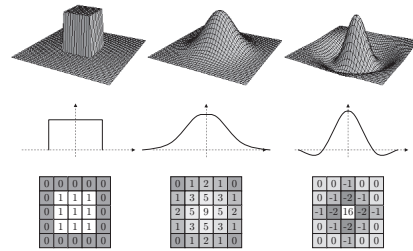
Weitere Beispiele

Glättungsfilter

- Box-Filter
- Gauß-Filter

Differenzfilter

- Laplace-Filter
- Positive und negative Werte



Lineare Filter & Faltung

- Bildverarbeitung (z.B. in Photoshop, TensorFlow)

$$I'(x, y) = \sum_{k=-K}^K \sum_{l=-L}^L H(k, l) \cdot I(x+k, y+l)$$

- Mathematik: Faltung *

$$\begin{aligned} (I * H)(x, y) &= \sum_{k=-K}^K \sum_{l=-L}^L H(k, l) \cdot I(x-k, y-l) \\ &= \sum_{k=-K}^K \sum_{l=-L}^L H(-k, -l) \cdot I(x+k, y+l) \end{aligned}$$

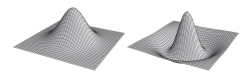
Lineare Filter & Faltung

- Definiere $H^*(k, l) = H(-k, -l)$

Beispiel: $\begin{bmatrix} 3 & 2 & 1 \\ 1 & 4 & 0 \\ 0 & 0 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 0 & 0 \\ 0 & 4 & 1 \\ 1 & 2 & 3 \end{bmatrix}$

- Oft: $H^* = H$

Beispiel: $\begin{bmatrix} 0 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 0 \end{bmatrix}$



- Dann: Linearer Filter & Faltung *identisch*

Faltung: Eigenschaften

- Kommutativität: $I * H = H * I$
- Linearität: $\begin{cases} (a \cdot I) * H = I * (a \cdot H) = a \cdot (I * H) \\ I * H_1 + I * H_2 = I * (H_1 + H_2) \end{cases}$
- Assoziativität: $(I * H) * G = I * (H * G)$

Lineare Filter: Separierbarkeit

- „Große“ Filter zerlegen in „kleine“ Filter
 - $H = H_1 * H_2 * \dots * H_n$
- „Kleine“ Filter hintereinander ausführen:
 - $I * H = (\dots((I * H_1) * H_2) * \dots * H_n)$
- Effekt: Einsparung von Rechenoperationen
 - Siehe nächste Folie

Lineare Filter: Separierbarkeit

□ Beispiel x-/y-Separation (Box-Filter):

$$H_{xy} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

3×5 = 15 Rechenoperationen pro Pixel

$$H_x = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad H_y = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

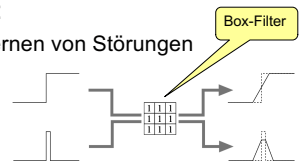
5+3 = 8 Rechenoperationen pro Pixel

$$(I * H_x) * H_y = I * (H_x * H_y) = I * (H_{xy})$$

Nichtlineare Filter

□ Lineare Filter: Nachteil:

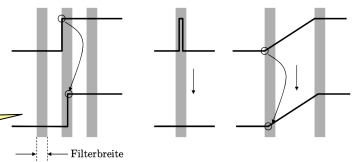
- Beim Glätten oder Entfernen von Störungen
 - Kanten & Linien werden verwischt
 - Geringere „Bildqualität“



□ Nichtlineare Filter

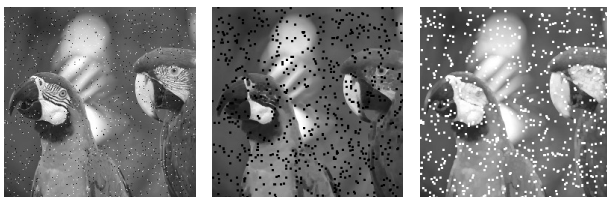
- Min- / Max-Filter
- Median-Filter

Bsp.: Minimum-Filter



Minimums- / Maximums-Filter

□ Test: Entfernen von „Salt & Pepper“ Rauschen



Gestörtes Originalbild

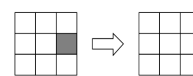
Minimum-Filter

Maximum-Filter

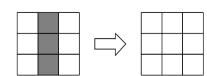
Median-Filter

□ Ersetze Pixel durch Median-Wert

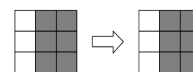
- Beispiel: 3 × 3 Median-Filter
- Auswirkung auf verschiedene Strukturen



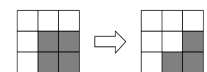
Einzelne Pixel eliminieren



Dünne Linien eliminieren



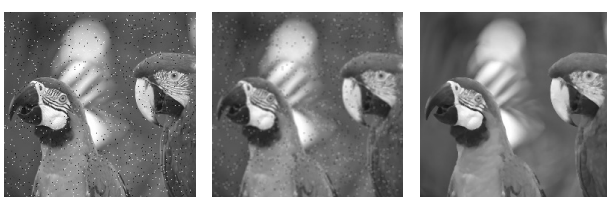
Kanten bleiben



Ecken abrunden

Median-Filter

□ Vergleich Glättungs- und Median-Filter



Gestörtes Originalbild

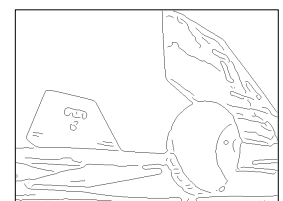
3×3 Box-Filter

Median-Filter

Kantendetektion

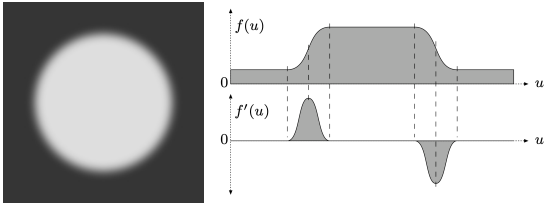
□ Visuelle menschliche Wahrnehmung

- Kanten haben besondere Bedeutung
- Objekte erkennen durch wenige Kanteninformationen



Gradienten

- Kanten dort, wo Betrag der Ableitung groß
- Kontinuierlich: $f'(u) = \frac{df}{du}(u)$



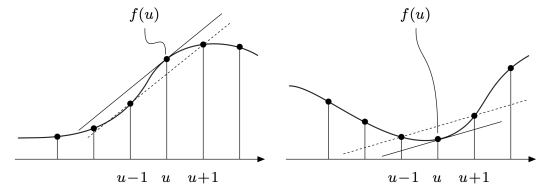
31

© Klaus Jung

Bildquelle: W. Burger, M. J. Burge: Digitale Bildverarbeitung

Gradienten

- Diskret: $\frac{df}{du}(u) \approx \frac{f(u+1) - f(u-1)}{2}$



32

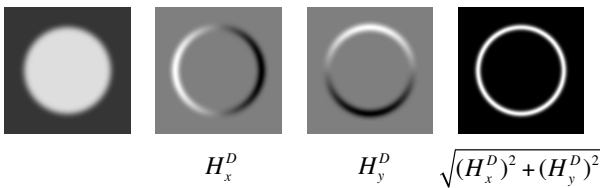
© Klaus Jung

Bildquelle: W. Burger, M. J. Burge: Digitale Bildverarbeitung

Ableitungsfilter

- Separiert in x- und y-Richtung

$$H_x^D = 0,5 \cdot \begin{bmatrix} -1 & 0 & 1 \end{bmatrix} \quad H_y^D = 0,5 \cdot \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}$$



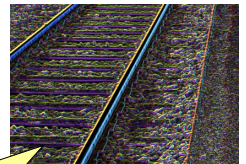
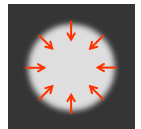
33

© Klaus Jung

Bildquelle: W. Burger, M. J. Burge: Digitale Bildverarbeitung

Gradient: Betrag und Richtung

- Gradient als Vektor
 - Betrag = Stärke der Änderung
 - Richtung der größten Änderung
- Berechnungen
 - x- und y-Komponente (g_x, g_y) mit H_x^D, H_y^D berechnen
 - Betrag $|g| = \sqrt{g_x^2 + g_y^2}$
 - Winkel berechnen mit Hilfe von $\arctan(g_y/g_x)$



34

© Klaus Jung

Betrag → Helligkeit / Winkel → Farbe

Bildquelle: K. Barthel

Prewitt-Filter

- Filter über 3 Zeilen / Spalten
 - Reduktion der Rausanfälligkeit

$$H_x^P = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad H_y^P = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

- Entspricht Ableitungsfilter gefolgt von 3x1 bzw. 1x3 Box-Filter zur Glättung

35

© Klaus Jung

Sobel-Filter

- Wie Prewitt-Filter, aber
 - Stärkere Gewichtung der zentralen Spalte / Zeile

$$H_x^S = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad H_y^S = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

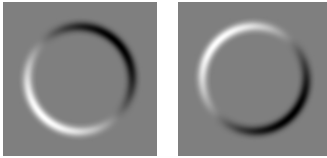
36

© Klaus Jung

Roberts-Filter

- Ableitungen in Diagonal-Richtung
 - Geringere Komplexität, da nur 2×2 -Kern

$$H_1^R = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \quad H_2^R = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$



37

© Klaus Jung

Bildquelle: W. Burger, M. J. Burge: Digitale Bildverarbeitung

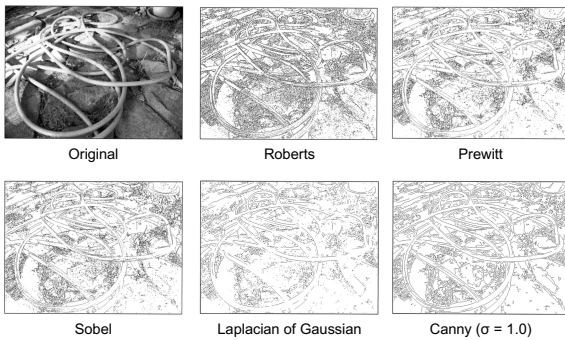
Zweite Ableitungen ausnutzen

- *Laplacian-of-Gaussian* (LoG) Operator
 - Zweite Ableitung = lokale Krümmung
 - Benutze Nulldurchgang zur Kantenlokalisierung
 - Anfällig für Rauschen → Glättung vorgeschaltet
- Canny-Filter
 - Kantendetektion auf verschiedenen Längenskalen
 - 3×3 -Filter erkennen nur scharfe Kanten innerhalb von 3 Pixel
 - Ziele
 - Falsche Markierungen minimieren
 - Kanten genau lokalisieren
 - Nur eine Markierung pro Kante

38

© Klaus Jung

Vergleich Kantendetektion

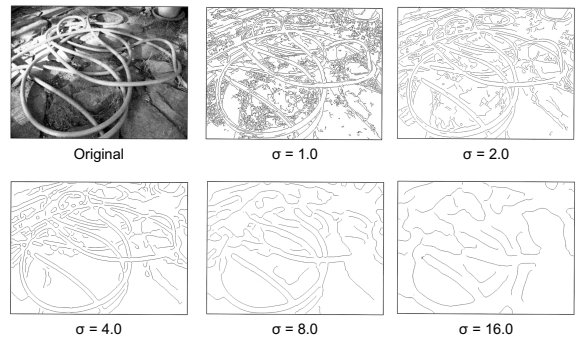


39

© Klaus Jung

Bildquelle: W. Burger, M. J. Burge: Digitale Bildverarbeitung

Canny Kanten-Detektion



40

© Klaus Jung

Bildquelle: W. Burger, M. J. Burge: Digitale Bildverarbeitung